

Missing Data - empty value, or a number 0, or an invalid value (a string for example) can be considered "missing data".

```
In [2]: import numpy as np
import pandas as pd
```

```
In [3]: falsy_values = (0, False, None, '', [], {})
```

For Python, all the values above are considered "falsy":

```
In [6]: any(falsy_values)  # "any" function return true if any value is true in an iterable
```

```
Out[6]: False
```

```
In [8]: np.nan  #Numpy has a special "nullable" value for numbers which is np.nan. It's N
```

```
Out[8]: nan
```

The np.nan value is kind of a virus. Everything that it touches becomes np.nan:

```
In [11]: print(3 + np.nan)

a = np.array([1, 2, 3, np.nan, np.nan, 4])
print(a.sum())

print(a.mean())
```

```
nan
nan
nan
```

```
In [12]: print(3 + None)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 print(3 + None)

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

```
In [14]: a = np.array([1, 2, 3, np.nan, None, 4], dtype='float')
print(a)

# As we said, np.nan is like a virus. If you have any nan value in an array and you
#perform an operation on it, you'll get unexpected results:

a = np.array([1, 2, 3, np.nan, np.nan, 4])

print(a.mean())
```

```
print(a.sum())

# Numpy also supports an "Infinite" type:
# np.inf
# Which also behaves as a virus:

print(3 + np.inf)
print(np.inf / 3)
print(np.inf / np.inf)
print(3 + np.nan)
```

```
[ 1.  2.  3. nan nan  4.]
nan
nan
inf
inf
nan
nan
```

```
In [16]: b = np.array([1, 2, 3, np.inf, np.nan, 4], dtype=float)
print(b.sum())
```

```
nan
```

```
In [ ]: ### It means numpy has "nan" & "inf" to display "Not a number" & "Infinite" resp.
```

Checking for nan or inf

```
In [17]: np.isnan(np.nan)
```

```
Out[17]: np.True_
```

```
In [18]: np.isinf(np.inf)
```

```
Out[18]: np.True_
```

And the joint operation can be performed with np.isfinite.

```
In [19]: np.isfinite(np.nan), np.isfinite(np.inf)
```

```
Out[19]: (np.False_, np.False_)
```

np.isnan and np.isinf also take arrays as inputs, and return boolean arrays as results:

```
In [21]: np.isnan(np.array([1, 2, 3, np.nan, np.inf, 4]))
```

```
Out[21]: array([False, False, False,  True, False, False])
```

```
In [22]: np.isinf(np.array([1, 2, 3, np.nan, np.inf, 4]))
```

```
Out[22]: array([False, False, False, False,  True, False])
```

```
In [23]: np.isfinite(np.array([1, 2, 3, np.nan, np.inf, 4]))
```

```
Out[23]: array([ True,  True,  True, False, False,  True])
```

Filtering out

```
In [26]: a = np.array([1, 2, 3, np.nan, np.nan, 4])
```

```
In [28]: np.isnan(a)
```

```
Out[28]: array([False, False, False,  True,  True, False])
```

```
In [27]: a[~np.isnan(a)] # Filtering out only not Nan
```

```
Out[27]: array([1., 2., 3., 4.])
```

```
In [29]: a[np.isfinite(a)].sum() # summing only which is finite numbers
```

```
Out[29]: np.float64(10.0)
```

```
In [31]: a[np.isfinite(a)].mean()
```

```
Out[31]: np.float64(2.5)
```
